

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи № 8 з дисципліни
«Основи програмування. Частина 2. Модульне програмування»

„ВІДНОШЕННЯ МІЖ КЛАСАМИ ТА ОБ'ЄКТАМИ”

Виконав(ла)

ІІ-56 Омелян Владислав Сергійович
(шифр, прізвище, ім'я, по батькові)

Перевірив

Куценко Микита Олександрович
(прізвище, ім'я, по батькові)

Київ 2026

Мета лабораторної роботи – дослідити типи відношень між класами та об'єктами в ООП, навчитися проектувати об'єктно-орієнтовану модель предметної галузі.

ЗАВДАННЯ

Варіант 7

Студентська бібліотека: облік літератури та читачьких формулярів

Функціональні вимоги до програмного забезпечення

1. Управління користувачами бібліотеки
 - 1.1. Можливість додавати користувачів
 - 1.2. Можливість видаляти користувачів
 - 1.3. Можливість змінювати дані користувачів
 - 1.4. Можливість переглянути дані конкретного користувача
 - 1.5. Можливість переглянути список всіх користувачів
 - 1.5.1. Можливість відсортувати список по імені
 - 1.5.2. Можливість відсортувати список по прізвищу
 - 1.5.3. Можливість відсортувати список по академічній групі
2. Управління документами (літературою) бібліотеки
 - 2.1. Можливість додавати документ
 - 2.2. Можливість видаляти документ
 - 2.3. Можливість змінювати дані документу
 - 2.4. Можливість переглянути дані конкретного документу
 - 2.5. Можливість переглянути список всіх документів
 - 2.5.1. Можливість відсортувати список по назві
 - 2.5.2. Можливість відсортувати список по автору
3. Управління видачами документів
 - 3.1. На рахунок користувача можна видавати n документів $n < 5$
 - 3.2. Можливість переглядати яку які документи взяв конкретний користувач
 - 3.3. Можливість по заданому документу визначити, чи він знаходиться у бібліотеці. Якщо документ виданий, то котрому користувачеві.
 - 3.4. Можливість повернути книжку в бібліотеку

Рис. 1 Умова завдання згідно мого варіанту

4. Пошук

4.1. Можливість пошуку по ключовому слову серед документів

4.2. Можливість пошуку по ключовому слову серед користувачів

Рис. 2 Продовження умови завдання згідно мого варіанту

Виконана робота розміщена на GitHub за посиланням:

https://github.com/VladOmelian/LAB_8_OOP.git

Приклад роботи

```
Бібліотеку ініціалізовано: 5 студентів, 5 документів.
```

```
СТУДЕНТСЬКА БІБЛІОТЕКА
```

1. Управління користувачами
2. Управління документами (літературою)
3. Управління видачами документів
4. Пошук
0. Вихід

```
-----  
Оберіть розділ: 1
```

```
--- Управління користувачами ---
```

1. Додати користувача (1.1)
2. Видалити користувача (1.2)
3. Змінити дані користувача (1.3)
4. Переглянути конкретного (1.4)
5. Список всіх користувачів (1.5)
0. Назад

```
Оберіть дію: 5
```

```
Сортувати за: 1-Ім'я 2-Прізвище 3-Група
```

```
Вибір [2]: 1
```

1. [Студент] Шаповал Артем, група: ІП-56, ID: 09c16186-cf52-4ab1-a9b2-bcc57bb5333e
2. [Студент] Омелян Владислав, група: ІП-56, ID: e9f78552-dffa-4e81-89e4-aa5616f49edc
3. [Студент] Петренко Ігор, група: ІМ-51, ID: acb99846-5547-4bb7-ab5f-c2aad91a571b
4. [Студент] Богомолів Олексій, група: ІК-51, ID: 4bde0226-6ea2-4319-add2-446611135a0b
5. [Студент] Мельник Роман, група: ІП-51, ID: 5cf21bda-035f-436a-b651-50b78238f09d

Рис. 3 Приклад роботи програми (сортування користувачів за ім'ям)

Діаграма класів мого ПЗ:

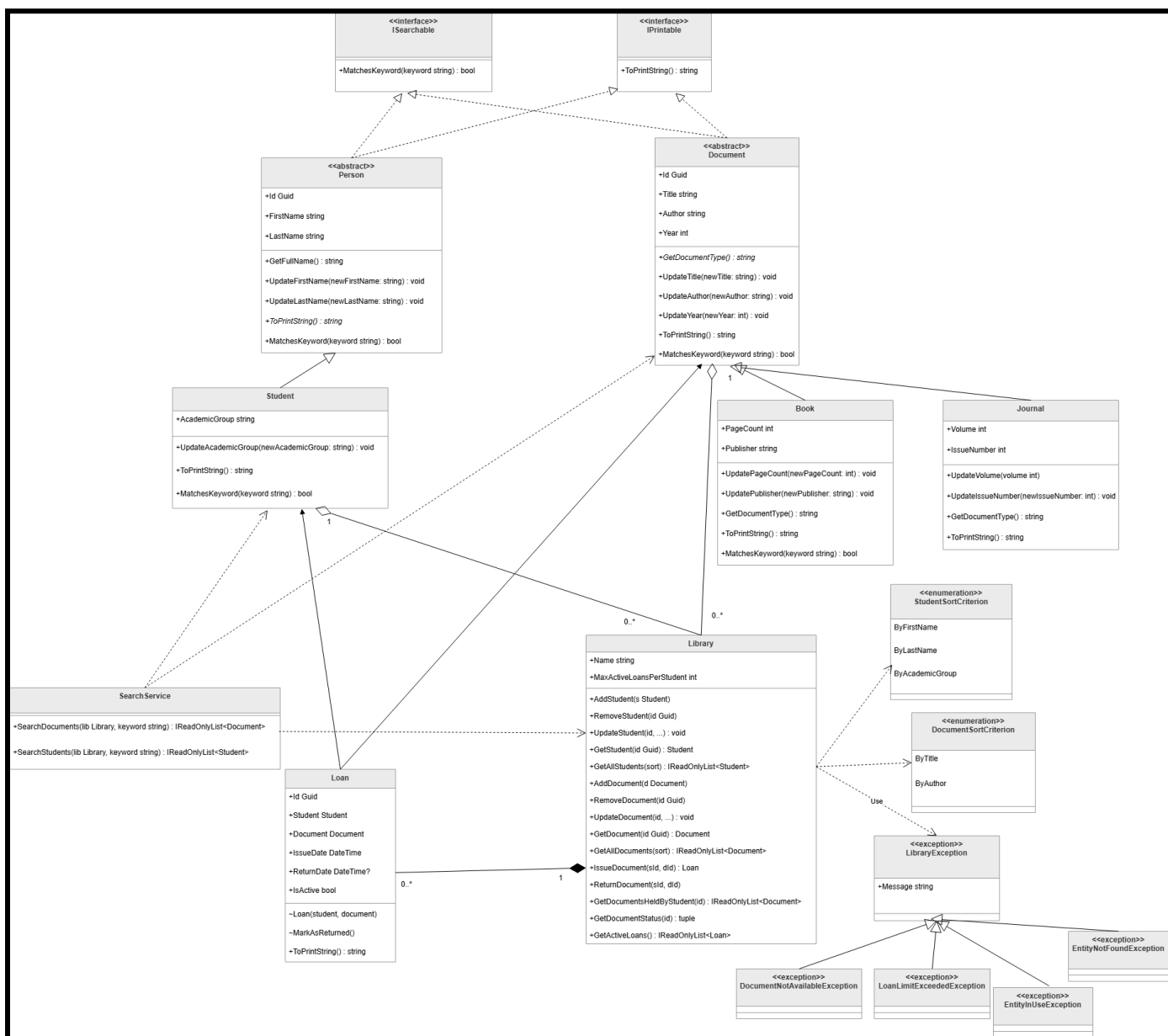


Рис. 4 Діаграма класів

Висновок: Під час виконання лабораторної роботи було досліджено типи відношень між класами та об'єктами в об'єктно-орієнтованому програмуванні та отримано практичні навички проектування об'єктно-орієнтованої моделі предметної галузі. Основна увага приділялася практичній реалізації зв'язків між класами: узагальнення застосовано при виділенні спільної логіки для постачальників та замовників у базовому класі; реалізація продемонстрована через інтерфейс іменованих сутностей; асоціація показана через незалежний зв'язок товару з постачальником. Агрегація успішно застосована у зв'язку між

категорією та списком товарів, тоді як композиція використана для опису характеристик товару, життєвий цикл яких повністю залежить від головного об'єкта. Крім того, на прикладі сервісу пошуку продемонстровано відношення залежності. Для контролю бізнес-логіки розроблено механізм обробки виняткових ситуацій із застосуванням власного класу помилок.

ВІДПОВІДІ НА КОНТРОЛЬНІ ПИТАННЯ

1. Перелічіть всі види відношень між типами та об'єктами

В об'єктно-орієнтованому проєктуванні (зокрема в UML) виділяють шість основних типів відношень:

1. **Залежність (Dependency)** – відношення "використовує" (Uses-a).
2. **Асоціація (Association)** – відношення "знає про" (Knows-a).
3. **Агрегація (Aggregation)** – слабке відношення "має" / "є частиною" (Has-a).
4. **Композиція (Composition)** – сильне відношення "є невід'ємною частиною" (Part-of).
5. **Наслідування / Узагальнення (Inheritance / Generalization)** – відношення "є" (Is-a).
6. **Реалізація (Realization)** – відношення реалізації інтерфейсу (Implements).

2. Наведіть приклади всіх відношень

Залежність (Dependency): Клас *SearchService* безпосередньо залежить від класу *Library*, оскільки приймає його як аргумент у методах, наприклад, *SearchDocuments(Library library, string keyword)*.

Асоціація (Association): Клас *Loan* знає про *Student* та *Document*, містячи посилання на ці об'єкти у своїх властивостях. Це рівноправний зв'язок взаємодіючих сутностей.

Агрегація (Aggregation): Клас *Library* володіє колекціями *students* та *documents*. Це відношення "ціле-частина", де частини можуть існувати самостійно.

Композиція (Composition): Клас *Library* керує колекцією *loans* (читацьких формулярів). Відповідні коментарі у коді також підкреслюють, що це саме композиція.

Наслідування (Inheritance): Класи *Book* та *Journal* узагальнюються базовим класом *Document*. Своєю чергою *Student* є нащадком *Person*.

Реалізація (Realization): Абстрактні класи *Document* та *Person* зобов'язуються реалізувати контракти інтерфейсів *IPrintable* та *ISearchable*.

3. Поясніть відмінності агрегації та композиції

Головна відмінність полягає у силі зв'язку та самостійності частин:

- Агрегація (слабкий зв'язок): У коді студенти (*Student*) та документи (*Document*) створюються десь ззовні (у клієнтському коді) і передаються в бібліотеку через методи *AddStudent* та *AddDocument*. Вони можуть спокійно існувати без об'єкта *Library*.
- Композиція (сильний зв'язок): Об'єкт *Loan* має конструктор з модифікатором доступу *internal*. Це фізично унеможлиблює його створення ззовні. Видачі (*Loan*) створюються лише всередині класу *Library* під час виклику методу *IssueDocument*. Життя видачі повністю залежить від бібліотеки.

4. Поясніть відмінності агрегації та наслідування

- Наслідування (Is-A): Використовується для розширення. Наприклад, *Student* є *Person*. Студент переймає поля *FirstName* і *LastName* з базового класу на етапі компіляції.
- Агрегація (Has-A): Використовується для збирання системи з готових компонентів. Бібліотека (*Library*) містить список документів, але вона ними не є.

5. Поясніть побічний ефект наслідування

Побічним ефектом наслідування є жорстка зв'язність класів та проблема «крихкого базового класу». У коді віртуальний метод *MatchesKeyword* реалізований у базовому класі *Document*. Клас *Book* перевизначає його (*override*), при цьому явно покладаючись на логіку предка через виклик *base.MatchesKeyword(keyword)*. Якщо я зміню поведінку цього методу в базовому класі *Document*, це автоматично і не завжди передбачувано вплине на те, як працює пошук для класів *Book* та *Journal*.

6. Поясніть життєвий цикл пов'язаних об'єктів у агрегації

У агрегації об'єкти мають незалежний життєвий цикл. Згідно з моєю логікою, якщо викликати метод *RemoveStudent(Guid studentId)*, студент буде

видалений з внутрішньої колекції `students` бібліотеки. Однак, сам екземпляр об'єкта `Student` не буде знищений програмою – він продовжить існувати в пам'яті клієнтського коду, де був початково створений.

7. Поясніть життєвий цикл пов'язаних об'єктів у композиції

У композиції об'єкти мають строго залежний життєвий цикл. Оскільки записи про видачу (`Loan`) створюються безпосередньо в екземплярі `Library` та зберігаються виключно в її приватній колекції `loans`, припинення існування об'єкта `Library` автоматично зробить колекцію `loans` недоступною. Усі об'єкти `Loan` будуть зібрані збирачем сміття (`Garbage Collector`) як такі, на які більше немає дійсних посилань.

8. Поясніть життєвий цикл пов'язаних об'єктів у наслідуванні

У наслідуванні життєвий цикл об'єкта працює за принципом матрошки (згори донизу). Це чудово видно в конструкторах класів. При створенні екземпляра `Student`, спочатку обов'язково викликається конструктор базового класу `Person` через `base(firstName, lastName)`. Це гарантує, що спочатку валідується та ініціалізується стан батька (генерується `Id`, присвоюється ім'я), і лише потім відбувається ініціалізація власного стану нащадка (зберігається `AcademicGroup`). В пам'яті це існує як єдиний, нерозривний блок даних.